Architectures logicielles et matérielles

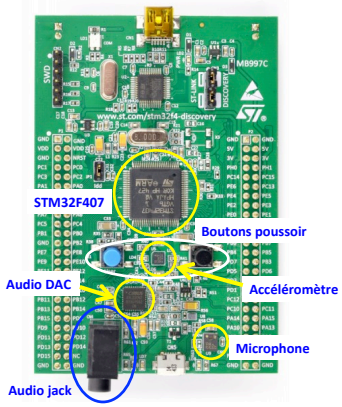
Cours 6 : Communications avec protocole SPI et I2C

Laurence Pierre

UGA Université Grenoble Alpes L3 Informatique 2026-2027

Introduction

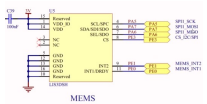
- Nous étudions ici les communications par interfaces SPI (Serial Peripheral Interface) et I2C (Inter-Integrated Circuit), très courantes dans l'embarqué
- En TP nous illustrerons SPI avec l'exemple de l'accéléromètre sur la carte STM32F4
  - Nous mentionnons aussi une utilisation de I2C dans le contexte de l'interface audio



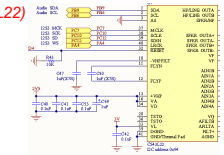
Introduction

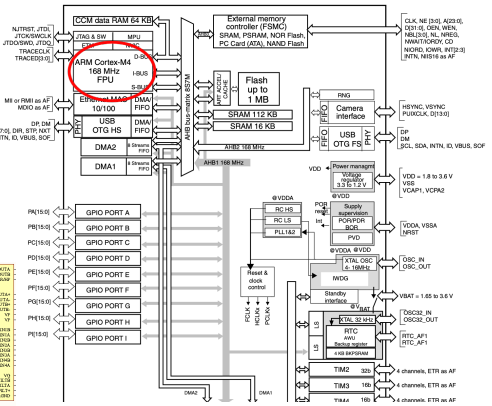
- Exemples de l'accéléromètre et de l'interface audio sur la carte STM32F4

Accéléromètre (LIS3DSH)
(interface SPI)



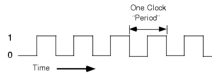
Audio-DAC (CS43L22)
(configuration par interface I2C)





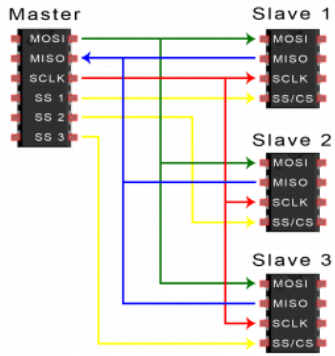
Communication par interface SPI

- Interface SPI = Serial Peripheral Interface
  - Développé par Motorola dans les années 1980
- Bus rapide pour la connexion maître-esclave(s) - 1 seul maître - avec des périphériques
  - Transmission synchrone de données "série", en **full duplex**
  - Utilise quatre signaux :
    - **SCLK** : horloge, générée par le maître
    - **SS** : signal Slave Select, généré par le maître (à 0 pour sélectionner un esclave)
    - **MOSI** : signal Master Output, Slave Input
    - **MISO** : signal Master Input, Slave Output



Communication par interface SPI

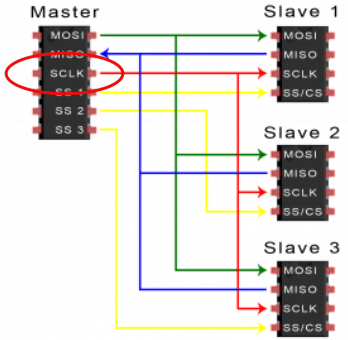
- Par exemple, avec 1 maître et 3 esclaves :



https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/

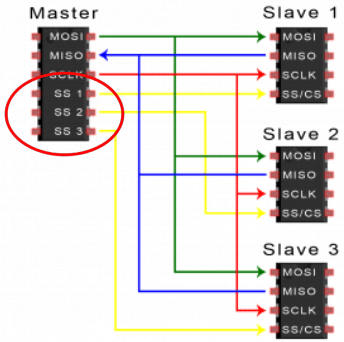
Communication par interface SPI

- Protocole SPI
  - Le maître **génère l'horloge** (SCLK) - la polarité et la phase peuvent en être configurées via un registre de contrôle (et la vitesse d'horloge est réglée selon les caractéristiques du périphérique esclave)



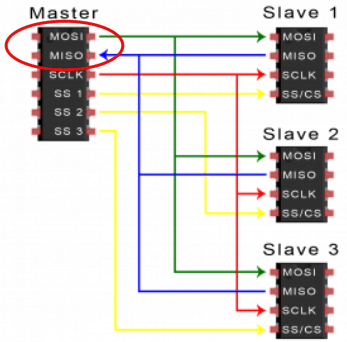
Communication par interface SPI

- Protocole SPI
  - Le maître **génère l'horloge** (SCLK) - la polarité et la phase peuvent en être configurées via un registre de contrôle (et la vitesse d'horloge est réglée selon les caractéristiques du périphérique esclave)
  - Il **sélectionne l'esclave** (SS, actif bas sur l'esclave) avec lequel il veut communiquer



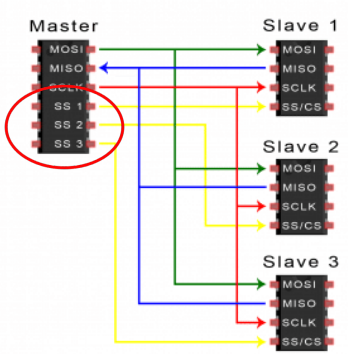
Communication par interface SPI

- Protocole SPI
  - L'esclave sélectionné réagira alors aux **requêtes** du maître
    - Les communications, en mode **full-duplex**, utilisent les lignes MOSI et MISO : à chaque cycle d'horloge, le maître et l'esclave **échangent 1 bit** (la ligne MOSI est utilisée pour les transmissions maître → esclave et la ligne MISO pour les transmissions esclave → maître)



Communication par interface SPI

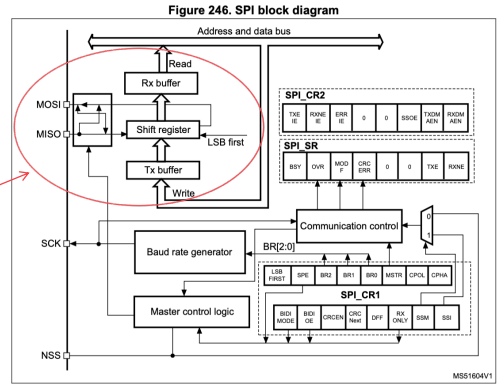
- Protocole SPI
  - L'esclave sélectionné réagira alors aux **requêtes** du maître
    - Les communications, en mode **full-duplex**, utilisent les lignes MOSI et MISO : à chaque cycle d'horloge, le maître et l'esclave **échangent 1 bit** (la ligne MOSI est utilisée pour les transmissions maître → esclave et la ligne MISO pour les transmissions esclave → maître)
  - A la fin du transfert, le maître **désélectionne** l'esclave



Communication par interface SPI

- Interface SPI
  - Exemple : block diagram de l'interface SPI sur la carte de TP

En réalité, les données envoyées et reçues passent par des buffers et sont "sérialisées" via un registre à décalage (1 bit émis/reçu par cycle d'horloge)



Communication par interface SPI

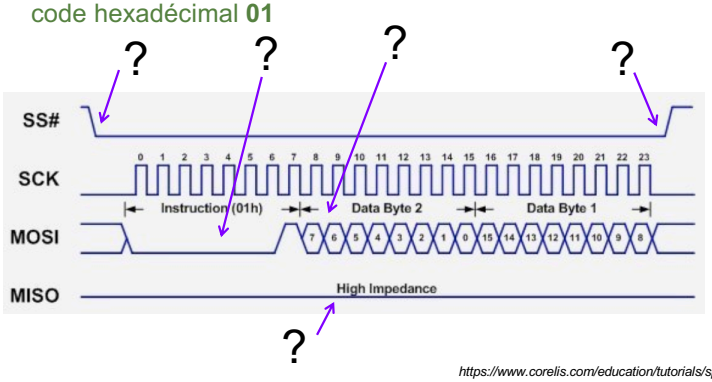
- Communication SPI
  - **Ecriture** de données
    - **Sélection** de l'esclave
    - Sur MOSI, envoi "instruction d'écriture" et **adresse** du registre dans lequel on veut écrire (dépend du contexte)

      Et lecture sur MISO des valeurs - non pertinentes - envoyées par l'esclave en full duplex (vidage tampon)
    - Puis envoi des **données** à écrire

      Et lecture sur MISO des valeurs - non pertinentes - envoyées par l'esclave en full duplex (vidage tampon)
    - **Désélection** de l'esclave

Communication par interface SPI

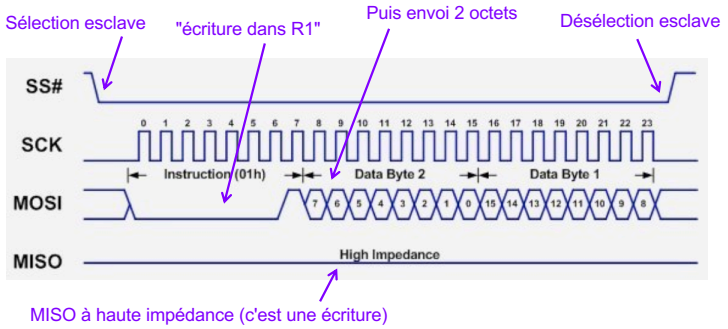
- Communication SPI - Ecriture
  - Exemple : Dans cet exemple (fictif), l'instruction d'écriture dans un registre donné de l'esclave choisi correspondrait au code hexadécimal **01**



https://www.corelis.com/education/tutorials/spi-tutorial/

Communication par interface SPI

- Communication SPI - Ecriture
  - Exemple : Dans cet exemple (fictif), l'instruction d'écriture dans un registre donné de l'esclave choisi correspondrait au code hexadécimal **01**



https://www.corelis.com/education/tutorials/spi-tutorial/

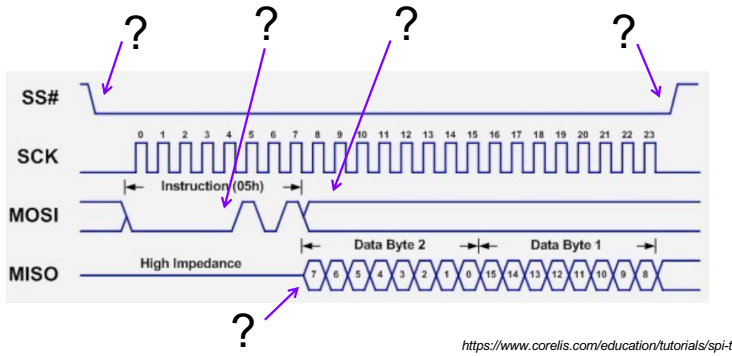
Communication par interface SPI

- Communication SPI
  - **Lecture** de données
    - **Sélection** de l'esclave
    - Sur MOSI, envoi "instruction de lecture" et **adresse** du registre dans lequel on veut lire (dépend du contexte)

      Et lecture sur MISO des valeurs - non pertinentes - envoyées par l'esclave en full duplex (vidage tampon)
    - Puis **réception** des données : *envoi sur MOSI de valeurs non pertinentes (pour pouvoir recevoir) et* lecture sur MISO des données envoyées par l'esclave en full duplex
    - **Désélection** de l'esclave

Communication par interface SPI

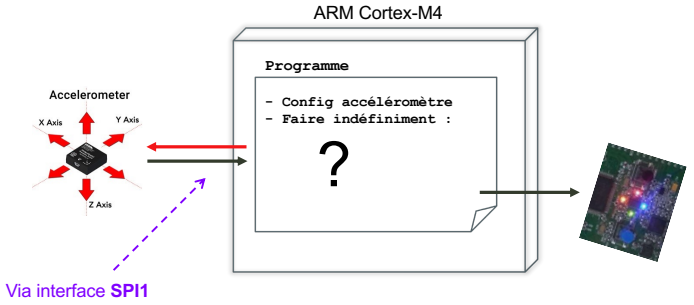
- Communication SPI - Lecture
  - Exemple : Dans cet exemple (fictif), l'instruction de lecture dans un registre donné de l'esclave choisi correspondrait au code hexadécimal **05**



https://www.corelis.com/education/tutorials/spi-tutorial/

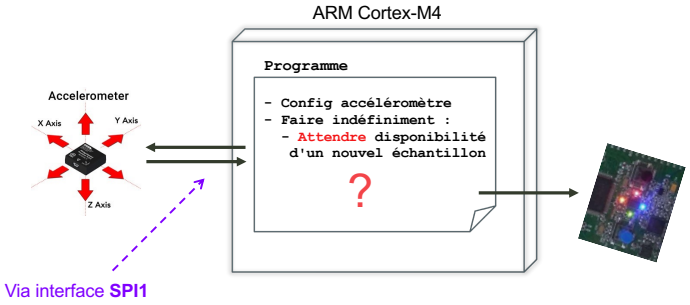
STM32F4 - Accéléromètre

- Application en TP
  - Configuration accéléromètre (LIS3DSH)
  - Puis acquisition échantillons successifs et allumage leds en conséquence



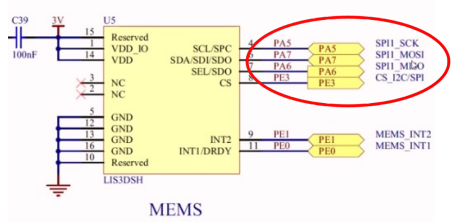
STM32F4 - Accéléromètre

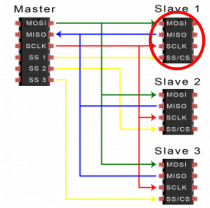
- Application en TP
  - Configuration accéléromètre (LIS3DSH)
  - Puis acquisition échantillons successifs et allumage leds en conséquence



STM32F4 - Accéléromètre

- Application en TP
  - Configuration accéléromètre (LIS3DSH)
  - Puis acquisition échantillons successifs
    - Via interface SPI1





Connexion aux ports GPIO :
MOSI (SDI) : port GPIOA, pin 7
MISO (SDO) : port GPIOA, pin 6
SCLK (SPC) : port GPIOA, pin 5
SS (CS) : port GPIOE, pin 3

STM32F4 - Accéléromètre

- Application en TP - Protocole SPI accéléromètre
  - **Ecriture** sur 16 cycles d'horloge
    - **Sélection** de l'esclave
    - Sur MOSI, bit d'écriture (valeur 0) suivi de l'adresse du registre sur 7 bits

      Et lecture sur MISO des valeurs - non pertinentes - envoyées par l'esclave en full duplex
    - Puis envoi des **données** sur 8 bits

      Et lecture sur MISO des valeurs - non pertinentes - envoyées par l'esclave en full duplex
    - **Désélection** de l'esclave

En **exercice** à préparer : étant donné add de type uint8_t, représentant une adresse, écrire l'expression formant 0 suivi des 7 bits de poids faible de add

STM32F4 - Accéléromètre

- Application en TP - Protocole SPI accéléromètre
  - **Lecture** sur 16 cycles d'horloge
    - **Sélection** de l'esclave
    - Sur MOSI, bit de lecture (valeur 1) suivi de l'adresse du registre sur 7 bits

      Et lecture sur MISO des valeurs - non pertinentes - envoyées par l'esclave en full duplex
    - Puis **réception** des données : *envoi sur MOSI de valeurs non pertinentes et* lecture sur MISO des données sur 8 bits envoyées par l'esclave en full duplex
    - **Désélection** de l'esclave

En **exercice** à préparer : étant donné add de type uint8_t, représentant une adresse, écrire l'expression formant 1 suivi des 7 bits de poids faible de add

STM32F4 - Accéléromètre

- Application en TP - Avant tout, **tester** nos fonctions de lecture et d'écriture
  - Test de la fonction de **lecture** ?

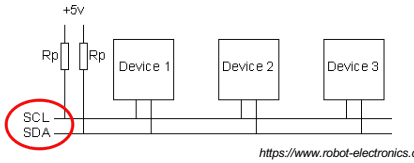
?

  - Test de la fonction d'**écriture** ? (<u>après</u> seulement)

?

Communication par interface I2C

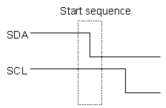
- Interface I2C = Inter-Integrated Circuit
  - Date également des années 1980, Philips (électronique domestique, téléviseurs)
  - Très simple et populaire, utilisé pour beaucoup de capteurs et actionneurs (mais moins bon débit que SPI)
- Bus série, multi-maître. Utilise seulement 2 signaux :
  - **SCL** : signal d'horloge, généré par le maître (Serial Clock)
  - **SDA** : signal pour les données (Serial Data)



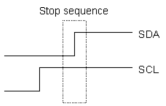
https://www.robot-electronics.co.uk/i2c-tutorial

Communication par interface I2C

- Protocole I2C
  - Emission d'une séquence "**start**" par un des maîtres pour initier un transfert



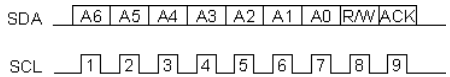
  - Puis adresses (sur 7 bits) et données seront transférées, le **maître** ayant l'accès au bus **génère le signal d'horloge**
  - A la fin du transfert, il utilise une séquence "**stop**"



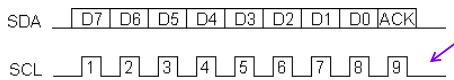
https://www.robot-electronics.co.uk/i2c-tutorial

Communication par interface I2C

- Protocole I2C
  - Adresses et données sont transférées par octet, le <u>récepteur</u> doit renvoyer un bit d'acknowledge pour chaque octet transféré (0 pour ACK, 1 pour NACK)
    - Transfert d'**adresse** : adresse sur 7 bits + 1 bit R/W (0 pour écriture, 1 pour lecture)



    - Transfert de **données** par blocs de 8 bits



Attention, 9 cycles d'horloge

https://www.robot-electronics.co.uk/i2c-tutorial

Communication par interface I2C

- Protocole I2C
  - Adresses et données sont transférées par octet, le récepteur doit renvoyer un bit d'acknowledge pour chaque octet transféré (0 pour ACK, 1 pour NACK)
    - Le NACK peut être utilisé pour diverses raisons
      - après l'envoi d'une adresse, pour indiquer que l'esclave à cette adresse ne peut pas répondre
      - après l'écriture d'une donnée, pour indiquer que l'écriture ne peut pas se faire, ou qu'on ne pourra plus accepter de données
      - lors de la lecture de données, pour que le maître indique à l'esclave qu'il ne voudra plus de nouvelles données
      - ...

https://www.robot-electronics.co.uk/i2c-tutorial

Communication par interface I2C

- Protocole I2C
  - **Ecriture** d'une donnée
    - Emission d'une séquence de "start"
    - Envoi de l'adresse de l'esclave vers lequel on veut faire l'écriture, avec le bit R/**W** à 0 / ACK
    - Envoi de l'adresse du registre dans lequel on veut écrire / ACK
    - Envoi des données, octet par octet / ACK
    - Emission de la séquence de "stop"
  - Exemple : **écriture** d'une donnée D (8 bits) dans un registre d'adresse A_0 d'un composant d'adresse A
    - Start - Envoi 7 bits d'adresse A + 1 bit R/**W** (à 0)
    - Envoi de l'adresse A_0
    - Envoi de la donnée D - Stop

Communication par interface I2C

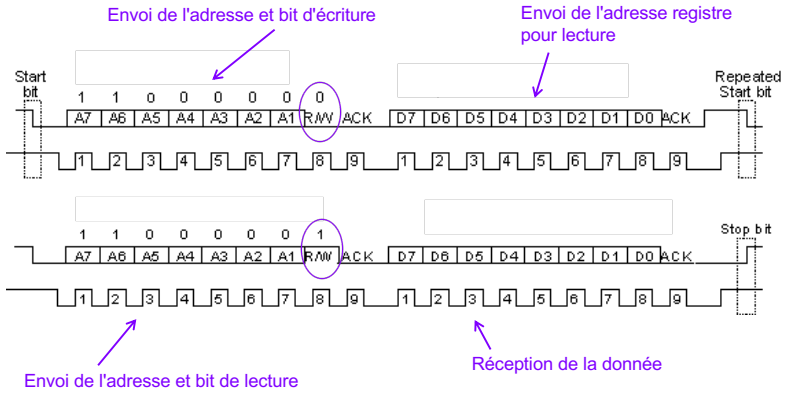
- Protocole I2C
  - **Lecture** d'une donnée
    - Emission d'une séquence de "start"
    - Envoi de l'adresse de l'esclave depuis lequel on veut faire la lecture, avec le bit R/**W** à 0 *(i.e., on va lui écrire l'adresse du registre)* / ACK
    - Envoi de l'adresse du registre dans lequel on veut lire / ACK
    - Nouvel envoi d'une séquence de "start"
    - Envoi de l'adresse de l'esclave depuis lequel on veut faire la lecture, avec le bit **R**/W à 1 *(i.e., on signifie la lecture)* / ACK
    - Lecture des données, octet par octet / ACK
    - Emission de la séquence de "stop"

Communication par interface I2C

- Protocole I2C
  - Exemple : **lecture** d'une donnée D (8 bits) dans un registre d'adresse A_0 d'un composant d'adresse A
    - Start
    - Envoi 7 bits d'adresse A + 1 bit R/**W** (à 0)
    - Envoi de l'adresse A_0
    - Start
    - Envoi 7 bits d'adresse A + 1 bit **R**/W (à 1)
    - Réception de la donnée D
    - Stop

Communication par interface I2C

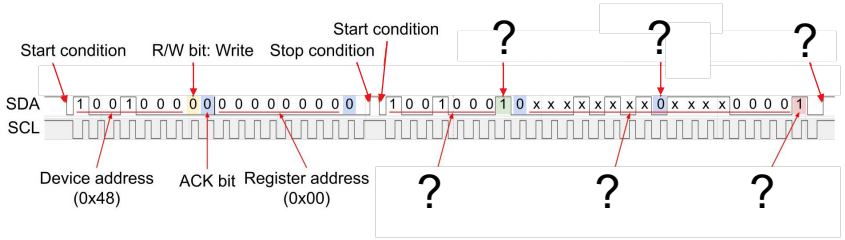
- Exemple 1



https://www.robot-electronics.co.uk/i2c-tutorial

Communication par interface I2C

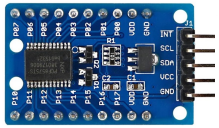
- Exemple 2

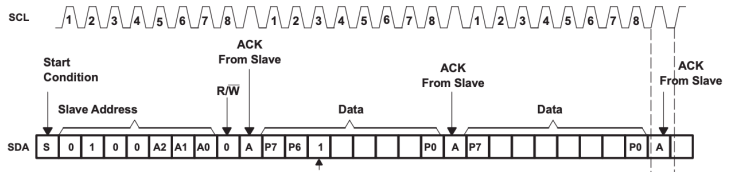


https://www.digikey.be/en/maker/projects/getting-started-with-stm32-i2c-example/ba8c2bfef2024654b5dd10012425fa23

Communication par interface I2C

- Remarque. Dans certains cas, il n'y a pas de transfert d'adresse d'un registre, seulement l'adresse de l'esclave
  - Exemple : Module d'extension I/O PCF8575, contrôlé par une interface I2C

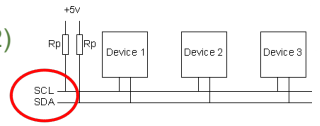


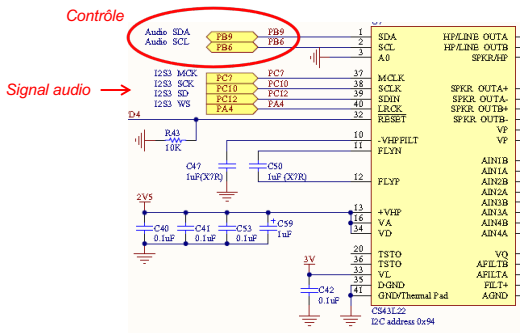


https://www.ti.com/lit/ds/scps123f/scps123f.pdf

STM32F4 - Configuration circuit audio

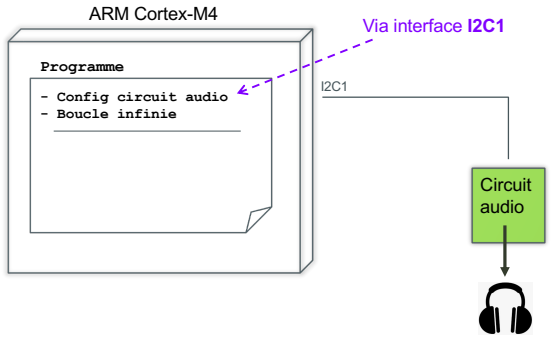
- Exemple d'utilisation
  - **Configuration** circuit audio (CS43L22)
    - Via interface I2C1





STM32F4 - Configuration circuit audio

- Exemple d'utilisation
  - **Configuration** circuit audio (CS43L22)
  - Puis envoi de donnée (signal audio), cadencé par TIMER2



STM32F4 - Configuration circuit audio

- Exemple d'utilisation
  - Configuration circuit audio (CS43L22)
  - Puis envoi de donnée (signal audio), cadencé par TIMER2

